

Práctica 5: Módulos Odoo

Controlador, Herencia y Web Controllers

Iván Alfaya Pérez

Fecha: 31 de enero de 2026

The Odoo logo, consisting of the word "odoo" in a white, lowercase, sans-serif font, centered on a dark purple rectangular background.

Índice

1. Introducción	2
2. Actividad 1: Modificar LigaFutbol	2
2.1. Reglas de Puntuación Especiales	2
2.1.1. Comprobación	4
2.2. Botones para Sumar Goles	5
2.2.1. Funciones suma de goles	5
2.2.2. Botones en la vista	5
2.3. Wizard para Actualizar Puntos	7
2.4. Web Controller Eliminar Empates	7
2.4.1. ¿Que es un web controller?	7
2.4.2. Código del web controller	7
2.5. Informe PDF por Partido	10
2.5.1. Plantilla del Informe	10
2.6. Wizard	13
2.6.1. ¿Que es un wizard?	14
2.6.2. Creación de los archivos	14
2.6.3. Código de los archivos	16
2.7. Vista Graph de Estadísticas	20
2.7.1. Configuración de la Vista	20
2.7.2. Código vista Graph	21
3. API REST	22
3.1. Vídeo Demostración	22
4. Bot de Telegram	23
4.1. Configuración con BotFather	23
4.2. Configuración del nuevo proyecto	27
4.3. Pruebas Bot telegram	31
5. Generador de imágenes aleatorias	38
5.1. 4.1 Instalar las librerías	38
5.2. 4.2 Estructura carpeta	40

1. Introducción

El objetivo de esta práctica es aplicar los conocimientos adquiridos sobre vistas, wizards, informes y controladores web. Probar una API REST mediante herramientas externas. Para ello, trabajarás en la implementación de cuatro actividades que abarcan diferentes contextos.

2. Actividad 1: Modificar LigaFutbol

Para esta parte modificaremos diferentes partes del modulo ya creado de LigaDeFutbol proporcionado en este enlace: [Enlace repositorio](#).

2.1. Reglas de Puntuación Especiales

Para ver el código anterior, consulta el [repositorio GitHub](#).

Modifique la funcion de puntos para que sea una variable normal y que no se actualice automaticamente, y en liga_partidos.py en el codifo que ya venia añadi la variable de puntos que de base es 0 y despues de comprobar si es una victoria, restar los goles_casa - goles_fuera para saber la diferencia y si es así sumar los puntos correspondientes y si no los puntos que venían por defecto y restar 1 punto al perdedor

```
record.jugados = record.victorias + record.empates + record.derrotas

#Los puntos los dejo en 0 y no se hacen automaticamente si no que lo hago en liga_partido.py
puntos = fields.Integer(default=0)

"""
puntos= fields.Integer( compute="_compute_puntos",default=0, store=True)

@api.depends('victorias','empates')
def _compute_puntos(self):
    for record in self:
        record.puntos = record.victorias * 3 + record.empates
"""

#Goles a favor y en contra
goles_a_favor=fields.Integer()
goles_en_contra=fields.Integer()
```

Figura 1: Código modificado en `liga_equipo.py`

```
def actualizoRegistrosEquipo(self):
    # Recorremos partidos y equipos
    for recordEquipo in self.env['liga.equipo'].search([]):
        # Como recalculamos todo, ponemos de cada equipo todo a cero
        recordEquipo.victorias = 0
        recordEquipo.empates = 0
        recordEquipo.derrotas = 0
        recordEquipo.goles_a_favor = 0
        recordEquipo.goles_en_contra = 0
        recordEquipo.puntos = 0

        for recordPartido in self.env['liga.partido'].search([]):
            # Si es el equipo de casa
            if recordPartido.equipo_casa.nombre == recordEquipo.nombre:
                # Miramos si es victoria o derrota
                if recordPartido.goles_casa > recordPartido.goles_fuera:
                    recordEquipo.victorias = recordEquipo.victorias + 1

                    # Reglas especiales, comprobar si hay diferencia de 4 en los goles, si no se aplica la puntuación por defecto
                    if (recordPartido.goles_casa - recordPartido.goles_fuera) >= 4:
                        recordEquipo.puntos = recordEquipo.puntos + 4
                    else:
                        recordEquipo.puntos = recordEquipo.puntos + 3

                # parte derrotas
                elif recordPartido.goles_casa < recordPartido.goles_fuera:
                    recordEquipo.derrotas = recordEquipo.derrotas + 1

                    if (recordPartido.goles_fuera - recordPartido.goles_casa) >= 4:
                        recordEquipo.puntos = recordEquipo.puntos - 1

            else:
                # Empate
                recordEquipo.empates = recordEquipo.empates + 1
                recordEquipo.puntos = recordEquipo.puntos + 1

            # Sumamos goles a favor y en contra
            recordEquipo.goles_a_favor = recordEquipo.goles_a_favor + recordPartido.goles_casa
            recordEquipo.goles_en_contra = recordEquipo.goles_en_contra + recordPartido.goles_fuera
```

Figura 2: regla especial en equipo local

```
# Si es el equipo de fuera
if recordPartido.equipo_fuera.nombre == recordEquipo.nombre:
    # Miramos si es victoria o derrota
    if recordPartido.goles_casa < recordPartido.goles_fuera:
        recordEquipo.victorias = recordEquipo.victorias + 1

        # lo mismo que lo anterior pero en el equipo de fuera o visitante
        if (recordPartido.goles_fuera - recordPartido.goles_casa) >= 4:
            recordEquipo.puntos = recordEquipo.puntos + 4
        else:
            recordEquipo.puntos = recordEquipo.puntos + 3

    elif recordPartido.goles_casa > recordPartido.goles_fuera:
        recordEquipo.derrotas = recordEquipo.derrotas + 1

        # verificar la derrota especial
        if (recordPartido.goles_casa - recordPartido.goles_fuera) >= 4:
            recordEquipo.puntos = recordEquipo.puntos - 1

    else:
        # Empate
        recordEquipo.empates = recordEquipo.empates + 1
        recordEquipo.puntos = recordEquipo.puntos + 1

    # Sumamos goles a favor y en contra
    recordEquipo.goles_a_favor = recordEquipo.goles_a_favor + recordPartido.goles_fuera
    recordEquipo.goles_en_contra = recordEquipo.goles_en_contra + recordPartido.goles_casa
```

Figura 3: regla especial en equipo visitante o fuera

2.1.1. Comprobación

Para la comprobación creare dos equipos como vemos en la imagen



Figura 4: Creación de equipos para prueba

Creacion del partido con diferencia de 4 goles o mas.

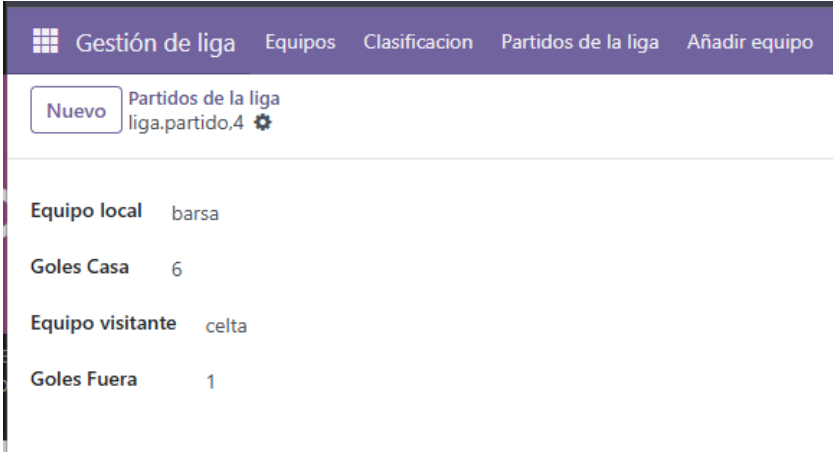


Figura 5: Creación partido con diferencia de goles

Resultado

Gestión de liga Equipos Clasificacion Partidos de la liga Añadir equipo									
Nuevo Clasificacion de la liga 1-2 / 2									
Buscar...									
Escudo equipo	Nombre equipo	Puntos	Jugados	Goles A F...	Goles En ...	Victorias	Empates	Derrotas	
	barsa	4	1	6	1	1	0	0	
	celta	-1	1	1	6	0	0	1	

Figura 6: Estadísticas de los equipos después del partido

2.2. Botones para Sumar Goles

Para esta parte añadiremos en los partidos dos botones que sumaran a todos los equipos locales o visitantes +2 goles

2.2.1. Funciones suma de goles

En el archivo liga_partido.py añadiremos las funciones de suma de goles

```
164 |  
165 |  
166 |  
167 | def sumar_2_goles_local(self):  
168 |     #coger todos los partidos sin filtro  
169 |     todos_partidos= self.search([])  
170 |     #recorrer cada equipo local y sumar +2 a la campo de goles_cada  
171 |     for cada_partido_local in todos_partidos:  
172 |         cada_partido_local.goles_casa+=2  
173 |  
174 |     # la funcion que actualiza todos los de los equipos y recalcula todo  
175 |     self.actualizoRegistrosEquipo()  
176 |  
177 | def sumar_2_goles_visitante(self):  
178 |     todos_partidos= self.search([])  
179 |     #recorrer cada equipo visistante y sumar +2 a la campo de goles_fuera  
180 |     for cada_partido_visitante in todos_partidos:  
181 |         cada_partido_visitante.goles_fuera+=2  
182 |  
183 |     # la funcion que actualiza todos los de los equipos y recalcula todo  
184 |     self.actualizoRegistrosEquipo()
```

Figura 7: Funciones para sumar 2 goles en local y visitante

2.2.2. Botones en la vista

Añadir los botones en liga_partido.xml tanto en la vista kanban como en la lista, en los botones en importante añadir la parte de "display=always" para que aparezca siempre por que si no lo pones no los veras

```

<!-- Vista LISTA -->
<record id="liga_partido_view_list" model="ir.ui.view">
  <field name="name">Lista de partidos de la liga</field>
  <field name="model">liga.partido</field>
  <field name="arch" type="xml">
    <list>
      <!-- botones de sumar goles en local y visitante en la vista de lista ,
           el always es importante añadirlo por que si no no te aparecieran los botones
      -->
      <header>
        <button
          name="sumar_2_goles_local"
          string="+2 goles local"
          type="object"
          display="always"
          confirm="¿Desea sumar 2 goles a los equipos locales?"/>
        <button
          name="sumar_2_goles_visitante"
          string="+2 goles visitante"
          type="object"
          display="always" ← IMPORTANTE!!!
          confirm="¿Desea sumar 2 goles a los equipos visitantes?"/>
      </header>
      <!-- Campos -->
      <field name="equipo_casa" />
      <field name="goles_casa" />
      <field name="equipo_fuera" />
      <field name="goles_fuera" />
    </list>
  </field>
</record>

```

Figura 8: Botones vista lista

```

<!-- Vista Kanban -->
<record id="liga_partido_view_kanban" model="ir.ui.view">
  <field name="name">Kanban Partidos</field>
  <field name="model">liga.partido</field>
  <field name="arch" type="xml">
    <kanban>
      <!-- botones de sumar goles en local y visitante en la vista de kanban -->
      <header>
        <button
          name="sumar_2_goles_local" ← Nombre de la funcion
          string="+2 goles local"
          type="object"
          display="always"
          confirm="¿Desea sumar 2 goles a los equipos locales?"
          class="oe_highlight"/>
        <button
          name="sumar_2_goles_visitante"
          string="+2 goles visitante"
          type="object"
          display="always"
          confirm="¿Desea sumar 2 goles a los equipos visitantes?"
          class="oe_highlight"/>
      </header>
    </kanban>
  </field>
</record>

```

Figura 9: Botones vista kanban

2.3. Wizard para Actualizar Puntos

2.4. Web Controller Eliminar Empates

Para esta parte crearemos un web controller para eliminar los partidos que estén en empates y devolver el numero de eliminados.

2.4.1. ¿Que es un web controller?

Un Web Controller en Odoo es una clase Python que maneja endpoints HTTP, exponiendo funcionalidades del modelo a través de URLs. Sirve como API REST o servidor web, permitiendo acceso externo a datos y operaciones sin la interfaz estándar de Odoo.

2.4.2. Código del web controller

Los web controller están en el archivo main.py en la carpeta controller del modulo.


```

28
29 #Se puede probar accediendo a http://localhost:8001/ligafutbol/eliminarempates
30 @http.route('/ligafutbol/eliminarempates', type='http', auth='none')
31 #funcion para eliminar los empates
32 def eliminarEmpates(self):
33     try:
34         #acceder al modelo odoo para los partidos
35         Partido = request.env['liga.partido']
36         #meter todos los partidos en una lista, lo que haria el sudo() sseria acceder a todos los datos
37         # ignorando los permisos, seria como la consulta de select * from tabla; (mostrar todos los datos de la tabla )
38         todos_partidos = Partido.sudo().search([])
39         #lista vacia para ir guardando los empates
40         lista_empates=[]
41         # info de los empates a eliminar para luego mostrar
42         lista_info_empates=[]
43
44         # recorrer todos los partidos
45         for partido in todos_partidos:
46             #estructura de la info del partido
47             partido_info = {
48                 "id": partido.id,
49                 "local": partido.equipo_casa.nombre,
50                 "goles_local": partido.goles_casa,
51                 "visitante": partido.equipo_fuera.nombre,
52                 "goles_visitante": partido.goles_fuera,
53                 "resultado": str(partido.goles_casa) + "-" + str(partido.goles_fuera),
54             }
55             #comprobar si los partidos son empates o no
56             if partido.goles_casa == partido.goles_fuera:
57                 #añadir a las listas
58                 lista_empates.append(partido)
59                 lista_info_empates.append(partido_info)
60
61             #total de empates
62             total_empates=len(lista_empates)
63
64             #comprobar si hay empates y mostrar la info
65             if total_empates> 0:
66                 #recorrer cada y eliminarlo con .unlink()
67                 for cada_empate in lista_empates:
68                     cada_empate.unlink()
69                 # respuesta si hay partidos empatados
70                 respuesta={
71                     "message": "Se han eliminado los partidos en empate",
72                     "total partidos eliminados": total_empates,
73                     "info partidos eliminados": lista_info_empates
74                 }
75             else:
76                 # respuesta en el caso de que no haya partidos para eliminar
77                 respuesta = {
78                     "message": "No hay partidos en empate para eliminar"
79                 }
80             # mandar la respuesta en formato json
81             return json.dumps(respuesta)
82
83
84
85 except Exception as e:
86     return json.dumps({"error": str(e)})
87

```

Figura 10: Código eliminar partidos

Creación de partidos en empate para las prueba:

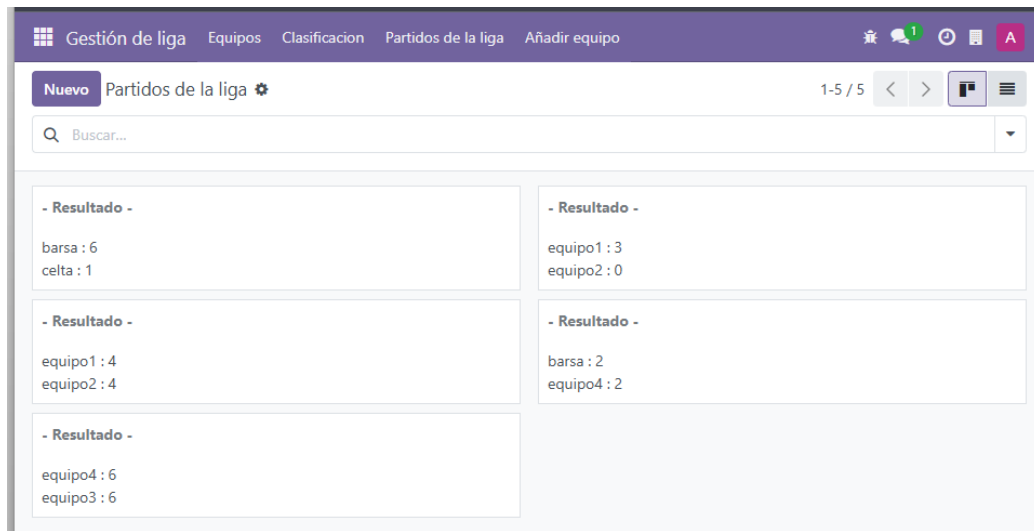


Figura 11: Creación partidos empate

Acceso a la url para borrar los partidos en empate,
<http://localhost:9001/ligafutbol/eliminarempates>

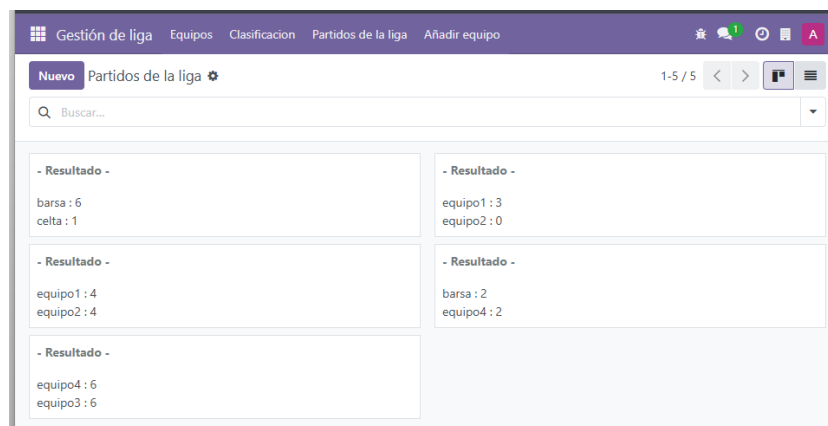


Figura 12: Acceso a la URL del controlador

Resultado:

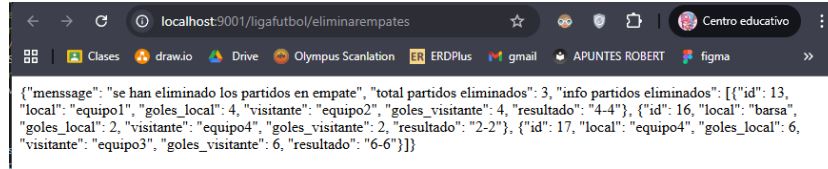


Figura 13: Respuesta de la eliminación

Vista en el modulo

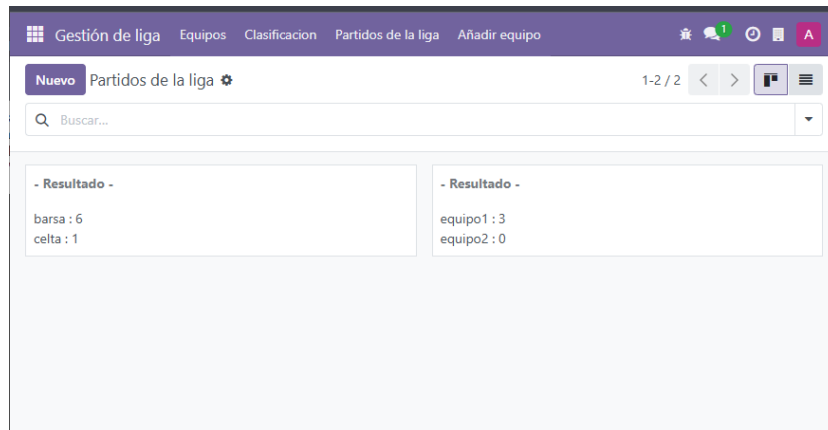


Figura 14: Vista odoo sin los empates

2.5. Informe PDF por Partido

Para esta parte tendremos que imprimir en formato pdf el partido

2.5.1. Plantilla del Informe

Para ello tendremos que crear un report que es un tipo de documento que le dice a odoo "genera este documento con estos datos y este estilo".

Ubicada en: `models/reports`

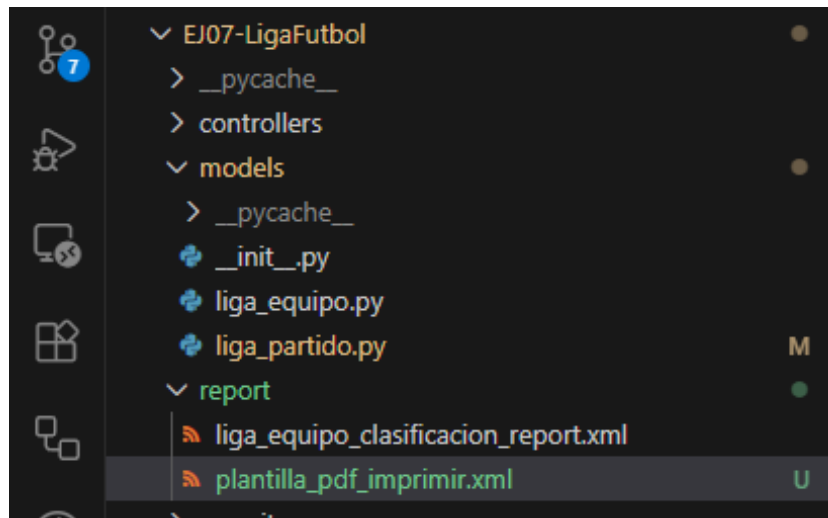


Figura 15: Archivo plantilla pdf

Creamos el archivo, en mi caso se llamara plantilla_pdf_imprimir, ojo, este archivo tenemos que añadirlo en el manifest del modulo para que lo reconozca.

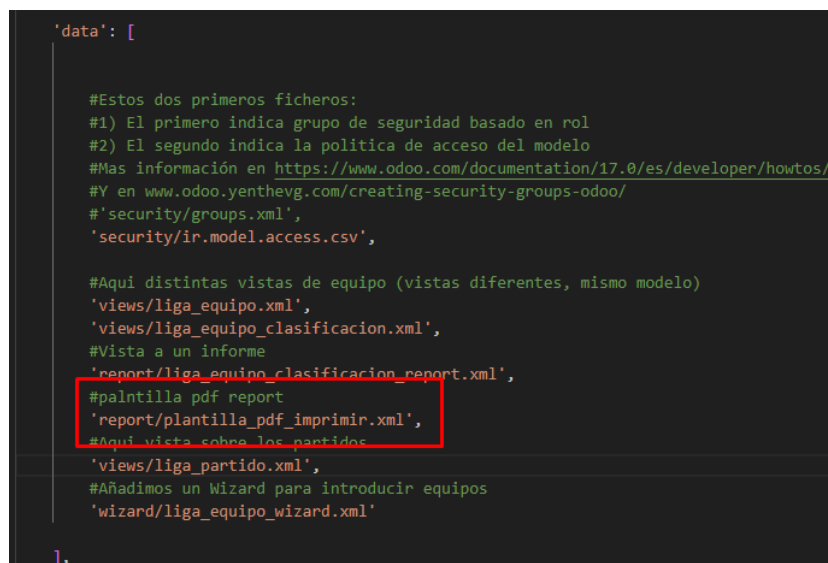


Figura 16: Inclusión en el manifest

También tendremos que añadir un botón para imprimir, en mi caso lo puse en la vista del formulario de los partidos en liga_partido.xml

```
<!-- VISTA DE FORMULARIO -->
<record id="liga_partido_view_form" model="ir.ui.view">
  <field name="name">Formulario Partidos</field>
  <field name="model">liga.partido</field>
  <field name="arch" type="xml">
    <form>
      <!-- boton para imprimir el pdf del partido -->
      <header>
        <button
          name="%(EJ07-LigaFutbol.boton_imprimir_pdf)d"
          string="Imprimir Informe"
          type="action"
          class="btn-primary"/>
      </header>
    </form>
  </field>
</record>
```

Figura 17: Botón para imprimir

Resultado:

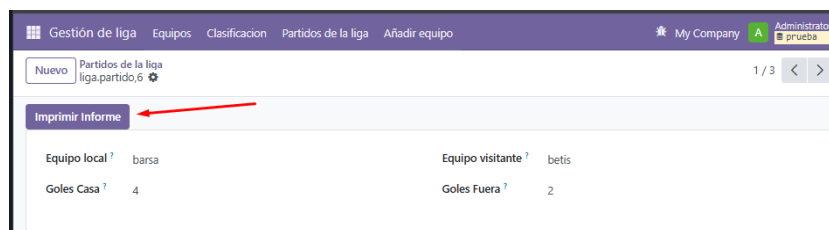


Figura 18: Acción botón

Se te descarga el documento y lo ves

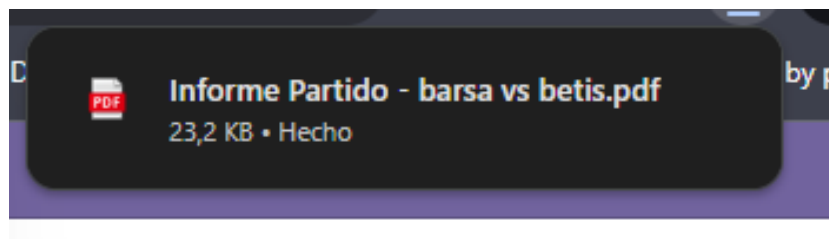
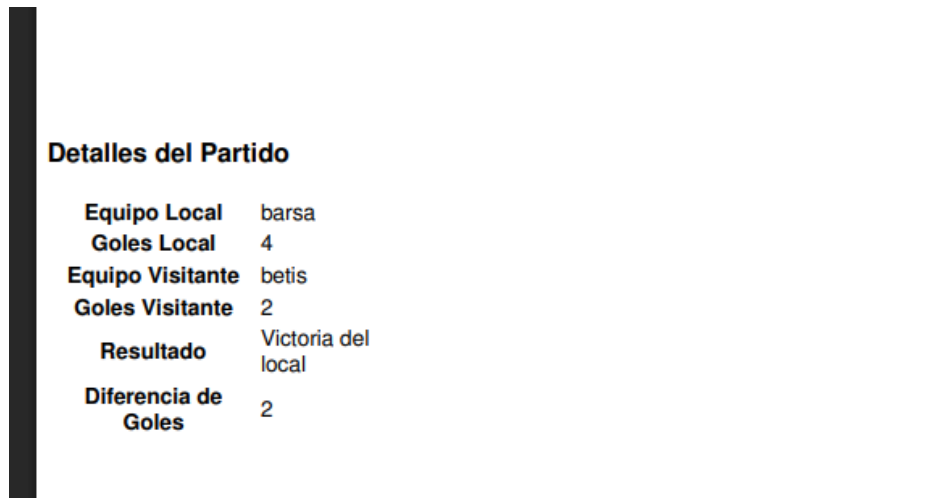


Figura 19: Documento pdf descargado



Detalles del Partido	
Equipo Local	barsa
Goles Local	4
Equipo Visitante	betis
Goles Visitante	2
Resultado	Victoria del local
Diferencia de Goles	2

Figura 20: Resultado visual

Estilo al documento.

Detalles del Partido	
Equipo Local	barsa
Goles Local	10
Equipo Visitante	betis
Goles Visitante	6
Resultado	Victoria del local
Diferencia de Goles	4

Figura 21: PDF con estilo

2.6. Wizard

Desarrollar un asistente interactivo que permita a los usuarios crear nuevos partidos de forma rápida y controlada, ingresando toda la información necesaria en una sola ventana emergente.

2.6.1. ¿Que es un wizard?

Un Wizard es una ventana emergente interactiva en Odoo que guía al usuario paso a paso para realizar tareas complejas, como si fuera un "asistente mágico" que te ayuda a completar procesos específicos.

Nota: Básicamente un modal de toda la vida por como lo entiendo

2.6.2. Creación de los archivos

Crearemos los archivos nuevos para la liga_partido.py y el xml en la carpeta wizard del modulo de liga de futbol

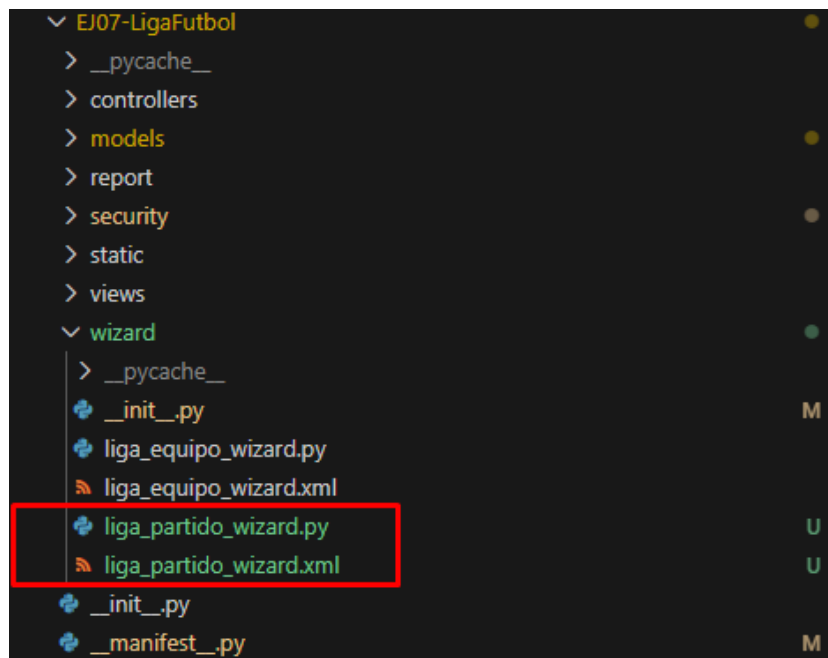


Figura 22: archivo.py y archivo.xml

Ya lo creamos ahora tenemos que añadirlo en el `_manifest_.py` y el `init`.

```
#Aqui vista sobre los partidos
'views/liga_partido.xml',
#Añadimos un Wizard para introducir equipos
'wizard/liga_equipo_wizard.xml',
#añadir el wizard de partido que hemos creado
'wizard/liga_partido_wizard.xml',

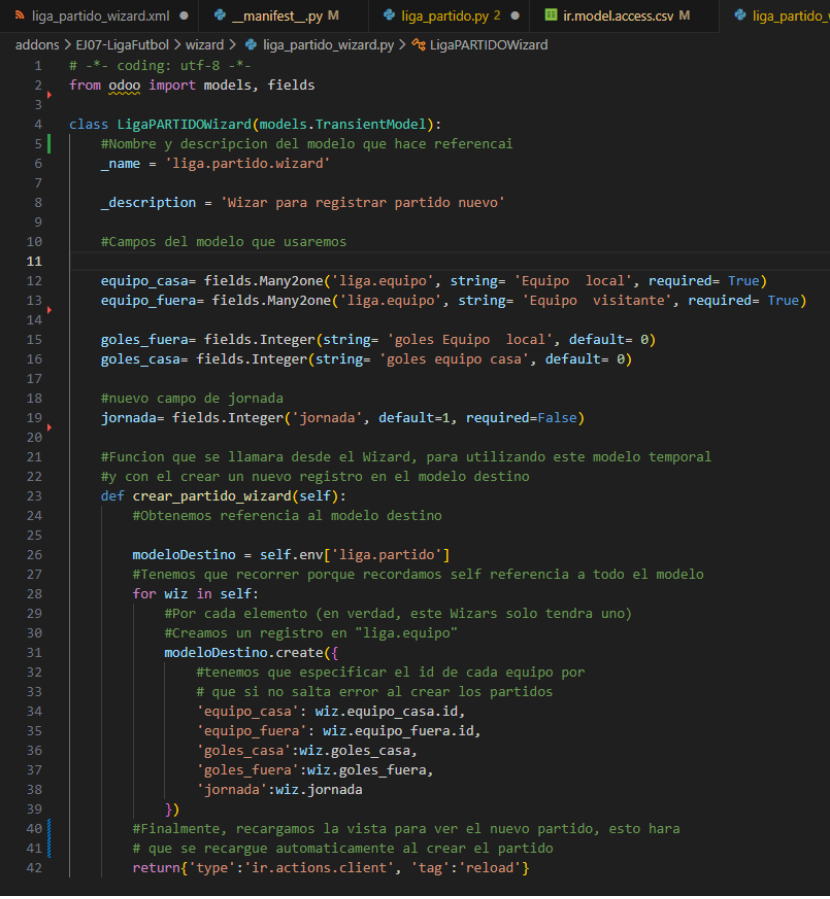
],
# Fichero con data de demo si se inicializa la base de datos con "de
# 'demo': [
#     'demo.xml'
# ],
}
```

Figura 23: añadir la vista en el manifest

```
__manifest__.py  __init__.py X  liga_equi
addons > EJ07-LigaFutbol > wizard > __init__.py
1  # -*- coding: utf-8 -*-
2  from . import liga_equipo_wizard
3  from . import liga_partido_wizard
4
```

Figura 24: añadir el modelo en el init

2.6.3. Código de los archivos

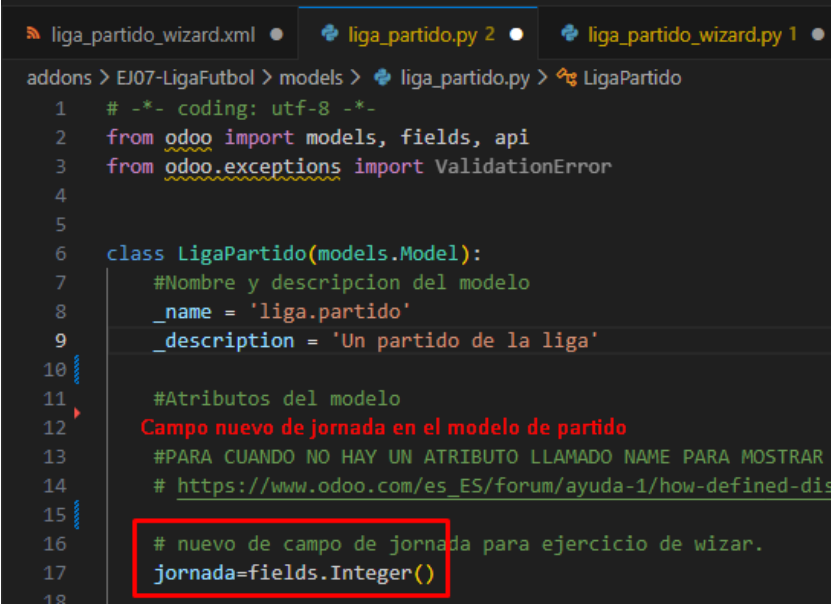


```
addons > EJ07-LigaFutbol > wizard > liga_partido_wizard.py > LigaPARTIDOWizard
1  # -*- coding: utf-8 -*-
2  from odoo import models, fields
3
4  class LigaPARTIDOWizard(models.TransientModel):
5      #Nombre y descripcion del modelo que hace referencia
6      _name = 'liga.partido.wizard'
7
8      _description = 'Wizar para registrar partido nuevo'
9
10     #Campos del modelo que usaremos
11
12     equipo_casa= fields.Many2one('liga.equipo', string= 'Equipo local', required= True)
13     equipo_fuera= fields.Many2one('liga.equipo', string= 'Equipo visitante', required= True)
14
15     goles_fuera= fields.Integer(string= 'goles Equipo local', default= 0)
16     goles_casa= fields.Integer(string= 'goles equipo casa', default= 0)
17
18     #nuevo campo de jornada
19     jornada= fields.Integer('jornada', default=1, required=False)
20
21     #Funcion que se llamara desde el Wizard, para utilizando este modelo temporal
22     #y con el crear un nuevo registro en el modelo destino
23     def crear_partido_wizard(self):
24         #Obtenemos referencia al modelo destino
25
26         modeloDestino = self.env['liga.partido']
27         #Tenemos que recorrer porque recordamos self referencia a todo el modelo
28         for wiz in self:
29             #Por cada elemento (en verdad, este Wizards solo tendra uno)
30             #Creamos un registro en "liga.equipo"
31             modeloDestino.create({
32                 #tenemos que especificar el id de cada equipo por
33                 # que si no salta error al crear los partidos
34                 'equipo_casa': wiz.equipo_casa.id,
35                 'equipo_fuera': wiz.equipo_fuera.id,
36                 'goles_casa':wiz.goles_casa,
37                 'goles_fuera':wiz.goles_fuera,
38                 'jornada':wiz.jornada
39             })
40
41         #Finalmente, recargamos la vista para ver el nuevo partido, esto hara
42         # que se recargue automaticamente al crear el partido
43         return {'type':'ir.actions.client', 'tag':'reload'}
```

Figura 25: código liga_partido_wizard.py

```
liga_partido_wizard.xml • liga_partido_wizard.py 1 •
addons > EJ07-LigaFutbol > wizard > liga_partido_wizard.xml
1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3 <!-- modificar el modelo referenciado -->
4 <record id="liga_partido_wizard_form" model="ir.ui.view">
5   <field name="name">Wizar nuevo partido</field>
6   <field name="model">liga.partido.wizard</field>
7   <field name="arch" type="xml">
8     <form string="Introducir datos de un partido">
9       <!-- modificar los campos-->
10
11       <group>
12         <group string="Equipos">
13           <field name="equipo_casa"/>
14           <field name="equipo_fuera"/>
15         </group>
16
17         <group string="Jornada y resultado">
18           <field name="goles_casa"/>
19           <field name="goles_fuera"/>
20           <field name="jornada"/>
21         </group>
22       </group>
23       <footer>
24         <!-- modificar el nombre de la funcion para crear partido -->
25         <button string="Añadir partido" name="crear_partido_wizard" class="btn-primary" type="object"/>
26         <button string="Cancel" class="btn-default" special="cancel"/>
27       </footer>
28     </form>
29   </field>
30 </record>
31
32 <!-- referenciar el id con el action del menu item de abajo-->
33 <record id="action_wizard_liga_partido" model="ir.actions.act_window">
34   <field name="name">Añadir Partido</field>
35   <field name="res_model">liga.partido.wizard</field>
36   <field name="view_mode">form</field>
37   <field name="target">new</field>
38 </record>
39
40
41 <menuitem id="menu_wizard_liga_partido" parent="liga_base_menu" action="action_wizard_liga_partido" sequence="20" />
42
43 </odoo>
```

Figura 26: código liga_partido_wizard.xml



```
liga_partido_wizard.xml ● liga_partido.py 2 ● liga_partido_wizard.py 1 ●
addons > EJ07-LigaFutbol > models > liga_partido.py > LigaPartido
1  # -*- coding: utf-8 -*-
2  from odoo import models, fields, api
3  from odoo.exceptions import ValidationError
4
5
6  class LigaPartido(models.Model):
7      #Nombre y descripcion del modelo
8      _name = 'liga.partido'
9      _description = 'Un partido de la liga'
10
11     #Atributos del modelo
12     Campo nuevo de jornada en el modelo de partido
13     #PARA CUANDO NO HAY UN ATRIBUTO LLAMADO NAME PARA MOSTRAR
14     # https://www.odoo.com/es\_ES/forum/ayuda-1/how-defined-dis
15
16     # nuevo de campo de jornada para ejercicio de wizar.
17     jornada=fields.Integer()
18
```

Figura 27: campo nuevo jornada en liga_partidopy

Gestión de liga Equipos Clasificación Partidos de la liga Añadir equipo Añadir Partido

Añadir Partido

SALTA EL MODAL Y RELLENAMOS DATOS DEL PARTIDIO CON EL NUEVO CAMPO

EQUIPOS

Equipo local	barça
Equipo visitante	madrid

JORNADA Y RESULTADO

goles equipo casa	31
goles Equipo local	1
jornada	2

Añadir partido Cancel

Figura 28: wizard partido

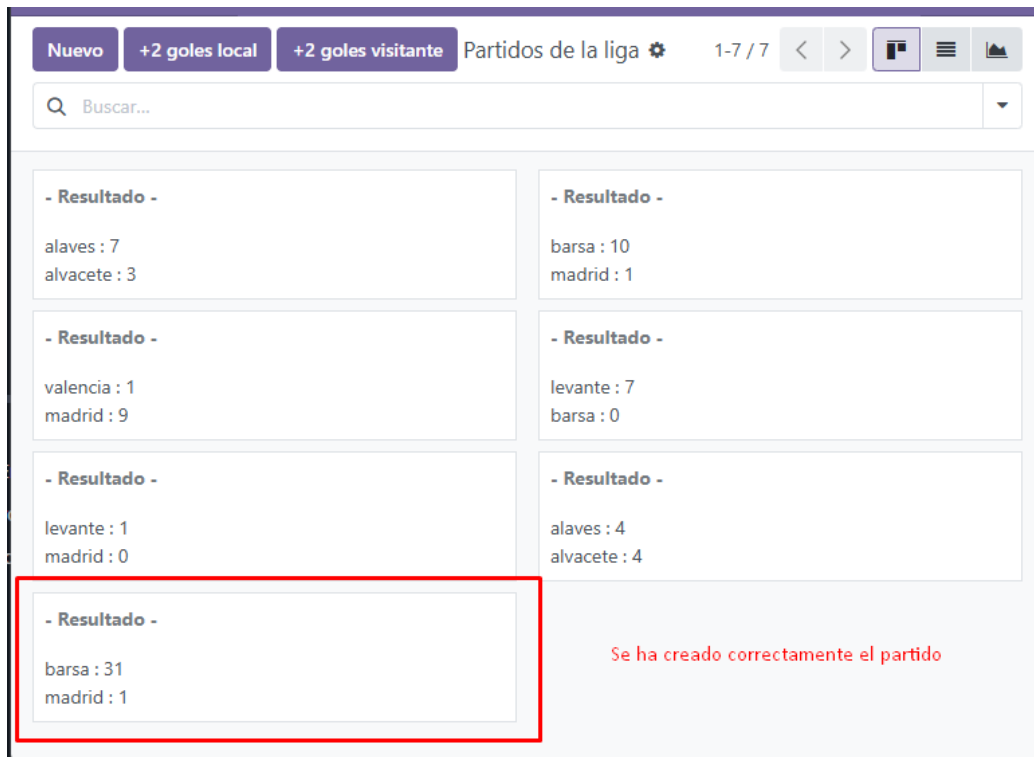


Figura 29: partido creado correctamente

2.7. Vista Graph de Estadísticas

Para esta parte crearemos una vista gráfica en `liga_partido.py` para mostrar los goles de los equipos locales clasificados por cada equipo

2.7.1. Configuración de la Vista

Para la configuración de la vista nos podemos basar en el graph ya hecho de `liga_equipo.py` y modificarlo

2.7.2. Código vista Graph

```

<record model="ir.ui.view" id="liga_partido_view_graph">
  <!-- este es un id interno para la vista de graf en la base de datos no es visible para el usuario en la interfaz -->
  <field name="name">vista grafica goles locales total</field>
  <!-- modelo al que hace referencia-->
  <field name="model">liga.partido</field>
  <!-- tipo -->
  <field name="type">graph</field>
  <field name="arch" type="xml">

    <graph string="Goles locales todos">
      <field name="equipo_casa" group="True" type="row"/>
      <!-- el type measure hace calculos del campo que le mandas por defecto hace una suma que lo que buscamos en este ejercicio-->
      <field name="goles_casa" group="True" type="measure" string="Total_goles_local"/>
    </graph>
  </field>
</record>

```

Figura 30: Graph de los partidos

Como vemos no es muy complicado las modificaciones que se hicieron fueron el modelo al que hace referencia, el nombre de los campos y lo que vera el usuario sera los goles locales metidos por cada equipo local, el campo de equipo_casa se agrupara por cada uno con sus goles es como hacer un select * from nombre_tabla group by campo.

Ojo: también añadir la vista del Graph arriba del todo para que aparezca

```

<?xml version="1.0" encoding="utf-8"?>
<odoo>

  <record id='liga_partido_action' model='ir.actions.act_window'>
    <field name="name">Partidos de la liga</field>
    <field name="res_model">liga.partido</field>
    <!-- añadir la parte de graph para poder alternar entre las vista -->
    <field name="view_mode">kanban,list,form,graph</field>
  </record>

  <menuitem name="Partidos de la liga" id="liga_partido_menu" parent="liga_base_menu" action="liga_partido_action" />
  <menuitem name="Partidos de la liga" id="liga_partido_menu" parent="liga_base_menu" action="liga_partido_action" />

```

Figura 31: añadir la vista Graph

quedaria asi:

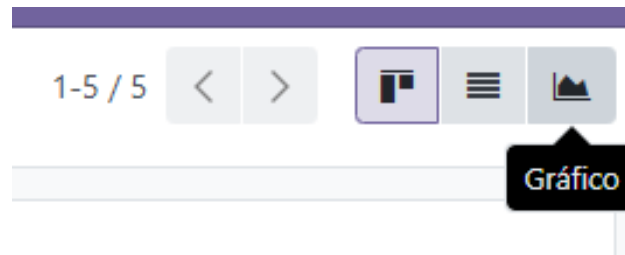


Figura 32: botón alternar vista

RESULTADO:

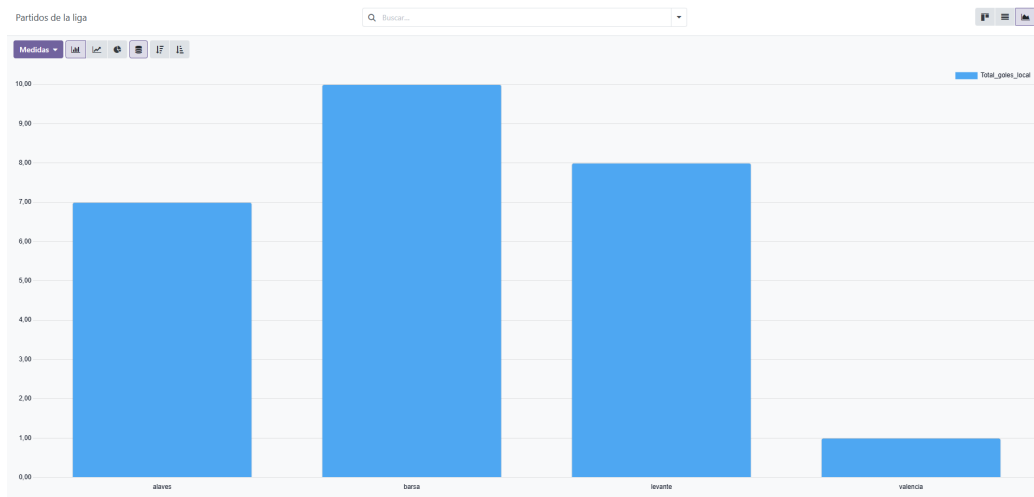


Figura 33: Vista Graph de los partidos

Como vemos se clasifican los partidos en funcion de su nombre con los goles que marcaron en local

3. API REST

3.1. Vídeo Demostración

En este video se probarán los diferentes endpoints de la API del módulo “EJ08-API-REST-Socios”, en donde se realizarán peticiones GET, POST, PUT y DELETE.

Nota sobre el formato del video: El video presenta una edición básica con subtítulos explicativos en lugar de narración vocal, por preferencias personales y prácticas de producción.

Aclaración importante: En una presentación previa se mencionó que se solicitó permiso para utilizar mis datos personales en las pruebas de esta API. **Debo aclarar que esta afirmación es incorrecta, ya que nunca se me solicitó tal autorización ni consentí el uso de mi información personal para este propósito. Un saludo**

Para ver el video pulsa [aquí](#)

4. Bot de Telegram

Para esta parte crearemos un bot de telegram con la api del ejercicio8 **EJ08-API-REST-Socios**, el modulo que importamos anteriormente para hacer el video.

4.1. Configuración con BotFather

1. Entramos en la app de telegram de tu móvil, y buscamos @BotFthater



Figura 34: buscar el botFather

2. Entrar al chat e iniciar /start (como vemos en la imagen cuando ponemos barra nos salen las diferentes opciones que puede a ver)

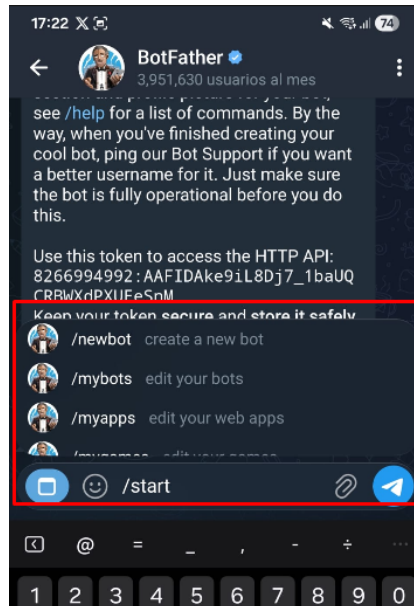


Figura 35: Entrar al chat e iniciar

3. Al iniciar nos manda un mensaje de pasos que podemos seguir como una ayuda, nos fijaremos la parte de /newbot para el siguiente paso.

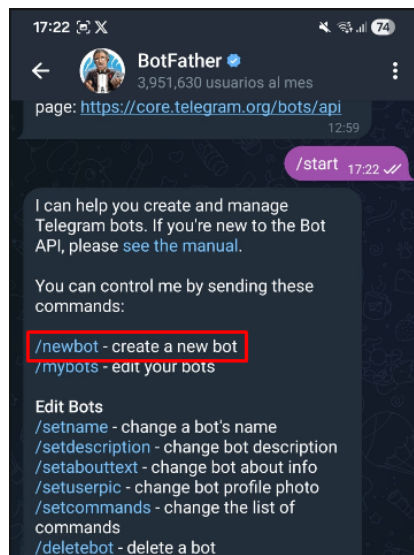


Figura 36: Entrar al chat e iniciar

4. Creamos un nuevo bot en donde le pondremos un nombre, y nos devolverá luego el enlace del bot y el token que lo usaremos mas adelante

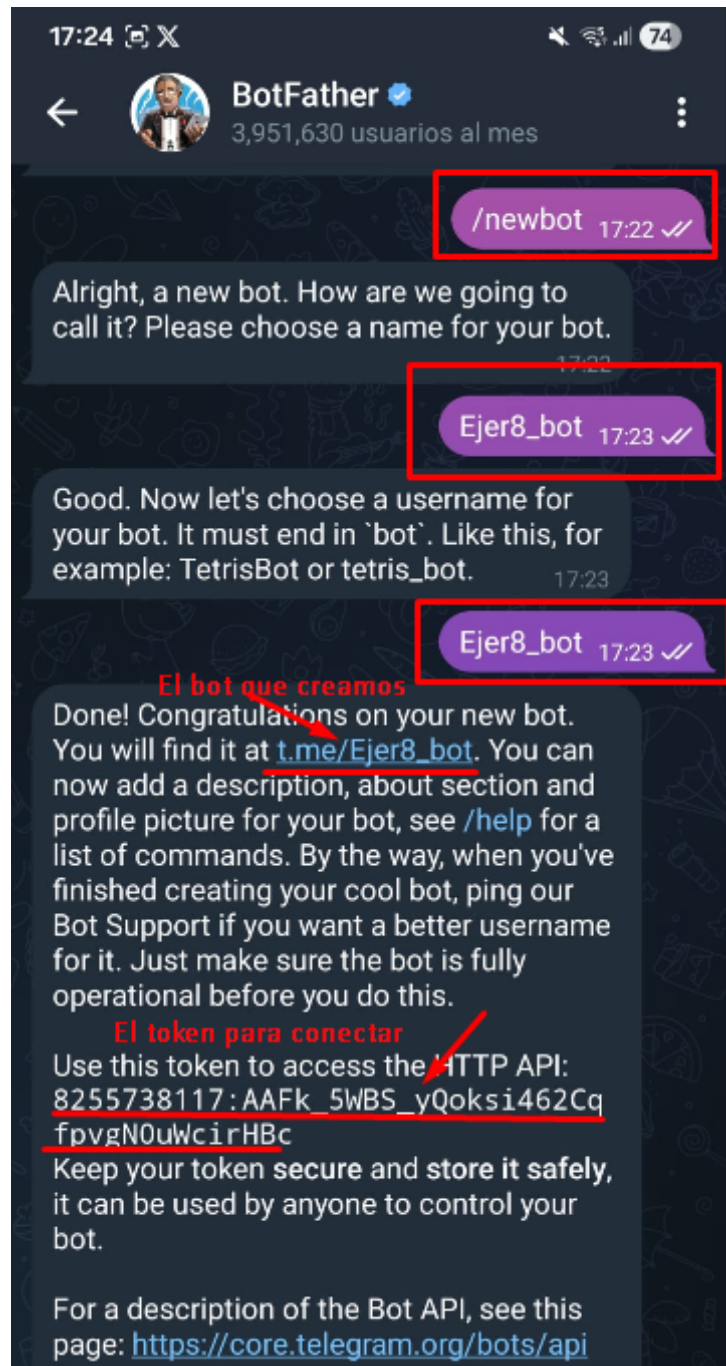


Figura 37: Entrar al chat e iniciar

5. Entrar en el chat del bot:

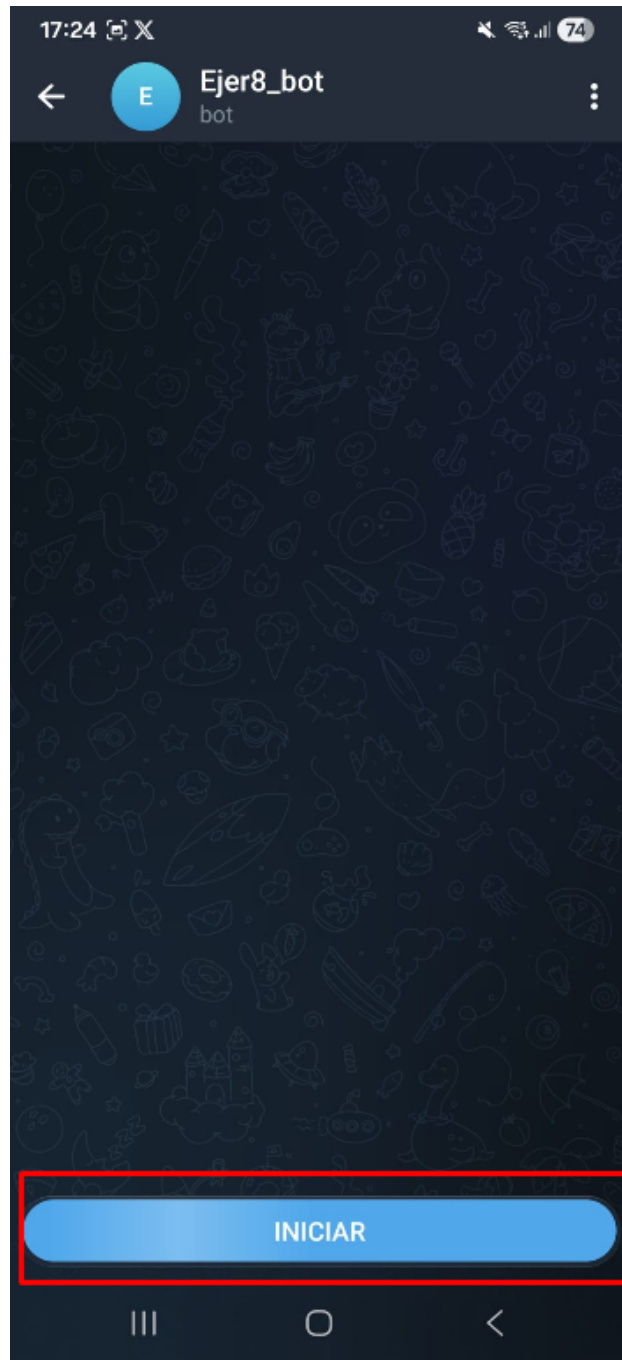


Figura 38: Entrar al chat e iniciar

Esto es para iniciar, pero ahora tenemos que configurar las ordenes que

recibira, vamos a ello.

4.2. Configuración del nuevo proyecto

Creacion de carpetas:

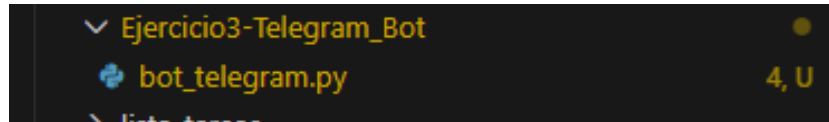


Figura 39: carpeta y proyecto que creare en la carpeta addons

Instalación de dependencias necesarias:

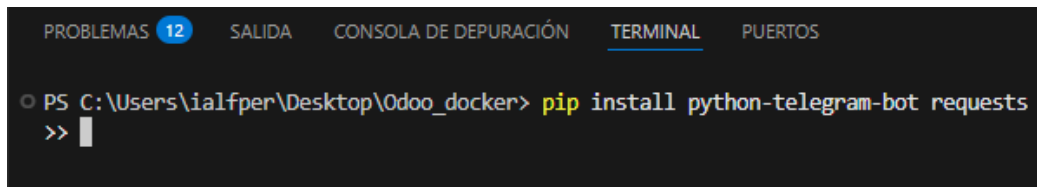


Figura 40: dependencias de telegram bot y request

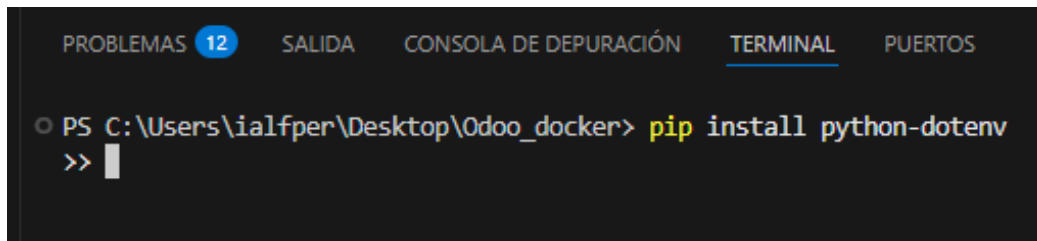
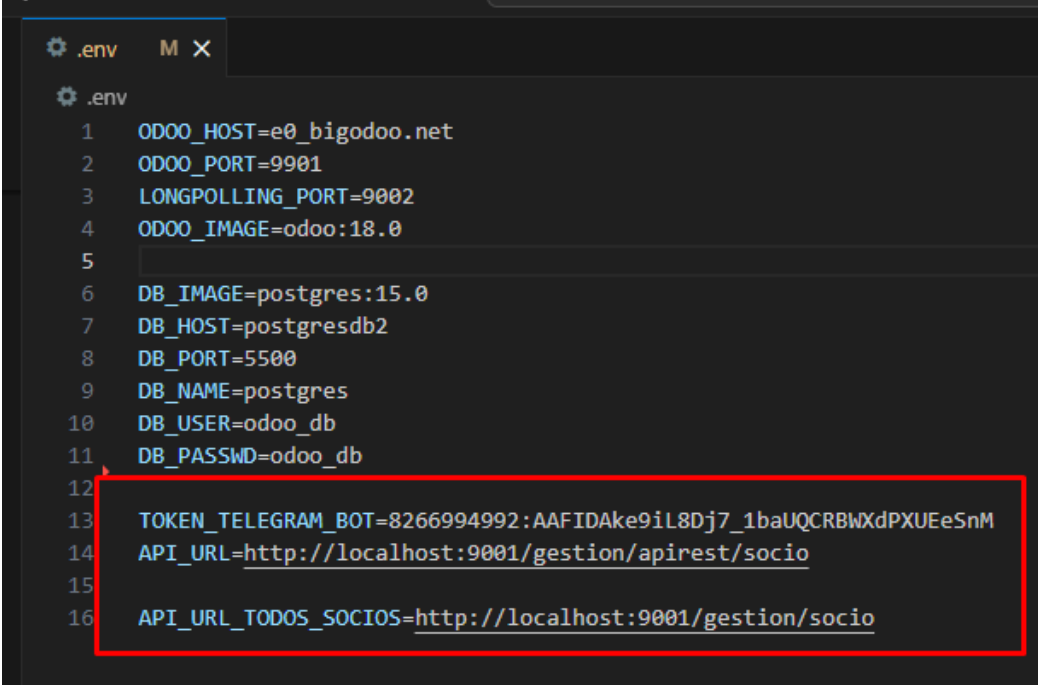


Figura 41: instalación de dotenv para usar la variables de entorno

Configuración variables de entorno



```
.env
1 ODOO_HOST=e0_bigodoo.net
2 ODOO_PORT=9901
3 LONGPOLLING_PORT=9002
4 ODOO_IMAGE=odoo:18.0
5
6 DB_IMAGE=postgres:15.0
7 DB_HOST=postgresdb2
8 DB_PORT=5500
9 DB_NAME=postgres
10 DB_USER=odoo_db
11 DB_PASSWORD=odoo_db
12
13 TOKEN_TELEGRAM_BOT=8266994992:AAFIDake9iL8Dj7_1baUQCRBwXdPXUEeSnM
14 API_URL=http://localhost:9001/gestion/apiREST/socio
15
16 API_URL_TODOS_SOCIOS=http://localhost:9001/gestion/socio
```

Figura 42: añadimos las variables de entorno del token, url de la api

!!EXTRA:!! también añadí la URL de la api en donde muestra todos los socios.

Use las variables de entorno por "Buenas prácticas", anteriormente las harcodeaba como se pueden ver en la imagen que son las que están comentadas.

```

addons > Ejercicio3-Telegram_Bot > bot_telegram.py > ...
1  # comando para instalar las libreria de request          pip install python-telegram-bot requests
2  # import de la librerias
3  from telegram import Update
4  from telegram.ext import ApplicationBuilder, CommandHandler, MessageHandler, ContextTypes, filters
5  import requests
6  import json
7  from dotenv import load_dotenv
8  import os
9
10 load_dotenv()
11 # el token que nos pasara el bot de telegram
12 #TOKEN = "8266994992:AAFIDake9il8Dj7_1baUQCRBwXdPXUEeSnM"
13 #la url de la api
14
15 #variables de entorno de la url de la api, de la mostrar todos y el token de telegram bot
16 API_URL=os.getenv("API_URL")
17
18 API_URL_TODOS_SOCIOS=os.getenv("API_URL_TODOS_SOCIOS")
19
20 TOKEN = os.getenv("TOKEN_TELEGRAM_BOT")
21
22 # API_URL = "http://localhost:9001/gestion/apirest/socio"
23 # API_URL_MOSTRAR_TODOS = "http://localhost:9001/gestion/socio"

```

Figura 43: parte de las importaciones de las librerías y las variables de entorno

```

# funcion para las ordenes
async def ordenes_comandos_telegram(update: Update, context: ContextTypes.DEFAULT_TYPE):
    #leera el mensaje completo, por ejemplo: "Crear, nombre="nombre",apellidos="apellidos", num_socio="numerosocio"
    mensaje_que_le_mandamos = update.message.text

    try:
        # vamos a dividirlo en partes
        partes = mensaje_que_le_mandamos.split(",") # cuando le mandamos el mensaje seria como el formato que le mandamos arriba, lo que haremos sera
        #segmentarlo por las comas "," que daría algo así : ["Crear, " nombre="Juan", "apellidos="Pérez", "num_socio="3"]

        # le indicamos que la orden es la posicion 0 osea la primera parte, usamos strip por si hay algun espacio para evitar problemas y pasarlo a minusculas
        orden = partes[0].strip().lower()

        # guardar los datos del usuarios, los campos y sus valores por ejemplo "nombre": "juan" y el resto si se lo mandamos
        datos_orden = {}

        # recorrer cada parte del resto de las instrucciones, pero empezamos a partir de la orden, se cogeria la parte de por ejemplo nombre= ivan y lo que siga
        for cada_parte in partes[1:]:
            tipo_orden, cada_campo = cada_parte.split("=")

            datos_orden[tipo_orden.strip()] = cada_campo.strip()

```

Figura 44: parte de recibir el mensaje y segmentar para las ordenes

```

# CREAR
if orden == "crear":
    print(datos_orden)
    datos_parseados=json.dumps(datos_orden)
    print("adasdjffffufuf"+datos_parseados)
    # con la libreria de requests le indicamos el tipo de petición que es, la parte de la url y los datos
    respuesta = requests.post(API_URL, datos_parseados)
    #mensajes para verlos en la terminal
    print(f"Se ha creado al usuario: {datos_orden}")

    # lo que te devuelve el bot de telegram con los que le hallas pasado
    await update.message.reply_text("Se ha creado al socio correctamente: "+ respuesta.text)

elif orden == 'todos':
    respuesta = requests.get(f"{API_URL TODOS SOCIOS}")
    print(f"Se muestran todos los usuarios OOKOKO: {respuesta}")
    await update.message.reply_text(respuesta.text)

```

Figura 45: segmentos de la diferentes ordenes

```

def main():
    app = ApplicationBuilder().token(TOKEN).build()

    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, ordenes_comandos_telegram))

    print("Bot SISISI")
    app.run_polling()

if __name__ == "__main__":
    main()

```

Figura 46: iniciar el proyecto

```

async def ayuda(update: Update, context: ContextTypes.DEFAULT_TYPE):
    mensaje_ayuda = """
    Lo que puedes hacer:

    • **Ver todos los socios**: todos
    • **Crear socio**: Crear, nombre=Juan, apellidos=fonso, num_socio=777
    • **Borrar socio**: borrar, num_socio= [ numero del socio ]
    • **Modificar socio**: put ,num_socio=[ numero del socio para modificar ], [ campo a modificar]= valor a modificar
    """
    await update.message.reply_text(mensaje_ayuda)

def main():
    Con el comand handler le indicamos que al poner "/" seguido del nombre llame a la funcion correspondiente
    app = ApplicationBuilder().token(TOKEN).build()

    #cuando hacer /help llama a la funcion que mostrara el mensaje de ayuda, esto es gracias al commandhandler que son todos los
    app.add_handler(CommandHandler("help", ayuda))

    # para el resto de las ordenes que le ponemos
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, ordenes_comandos_telegram))
    #mensaje en la terminal del proyecto para ver si inicio correctamente
    print("Bot SISISI")

    app.run_polling()

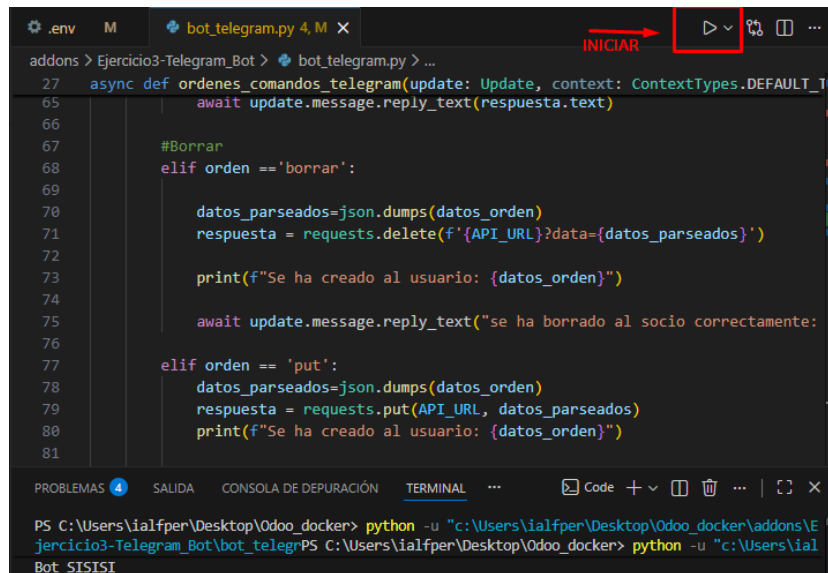
if __name__ == "__main__":
    main()

```

Figura 47: función de ayuda y llamada para iniciar

4.3. Pruebas Bot telegram

Iniciamos el proyecto:



```
27 async def ordenes_comandos_telegram(update: Update, context: ContextTypes.DEFAULT_T
65     await update.message.reply_text(respuesta.text)
66
67     #Borrar
68     elif orden == 'borrar':
69
70         datos_parseados=json.dumps(datos_orden)
71         respuesta = requests.delete(f'{API_URL}?data={datos_parseados}')
72
73         print(f"Se ha creado al usuario: {datos_orden}")
74
75         await update.message.reply_text("se ha borrado al socio correctamente: ")
76
77     elif orden == 'put':
78         datos_parseados=json.dumps(datos_orden)
79         respuesta = requests.put(API_URL, datos_parseados)
80         print(f"Se ha creado al usuario: {datos_orden}")
81
```

Figura 48: iniciar el proyecto

Entramos en bot de telegram para probar:

1. Volvemos al chat de telegram y probamos /help

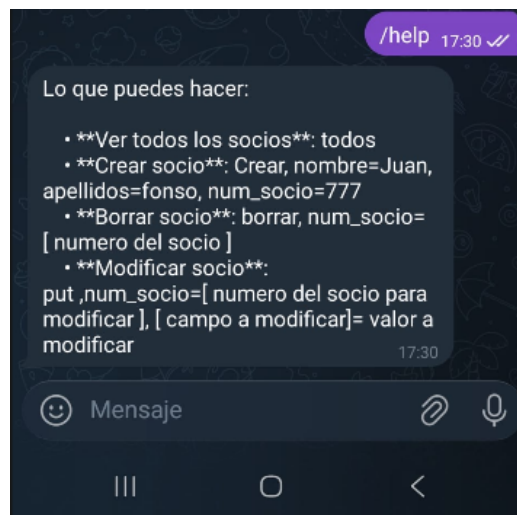


Figura 49: comando help

2. Ver todos los socios con "todos".

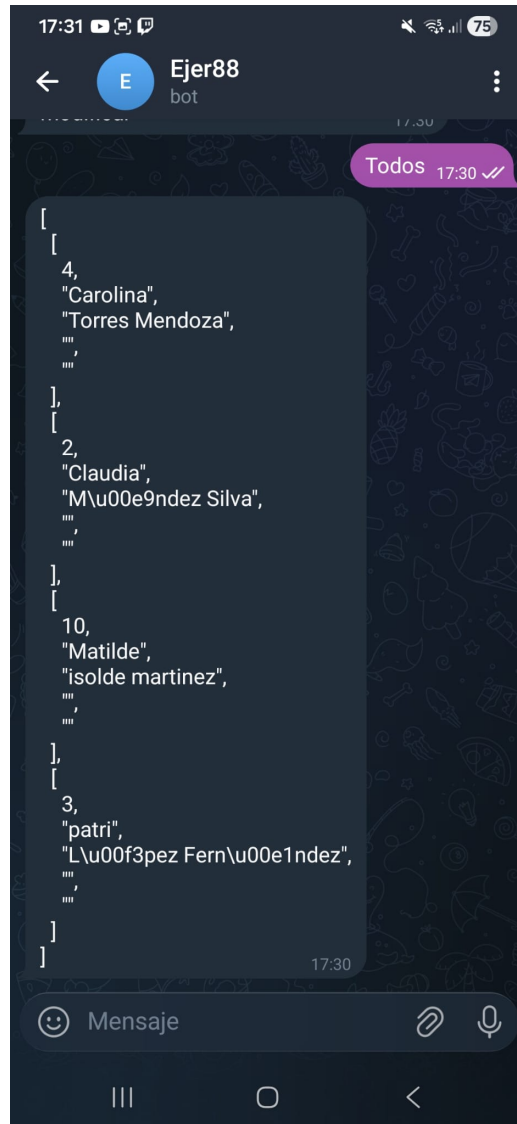


Figura 50: comando help

3. Creacion de un nuevo socio:

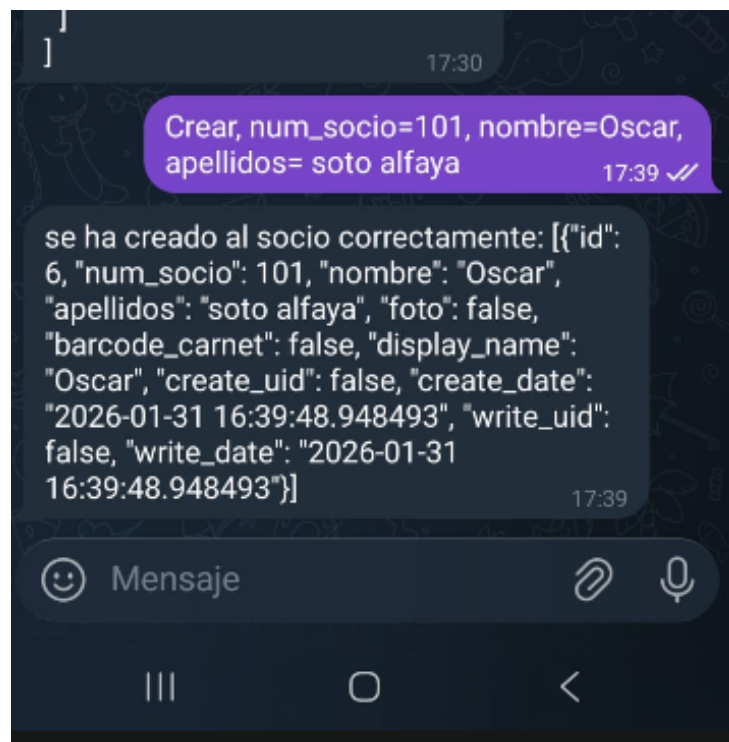


Figura 51: creación nuevo socio

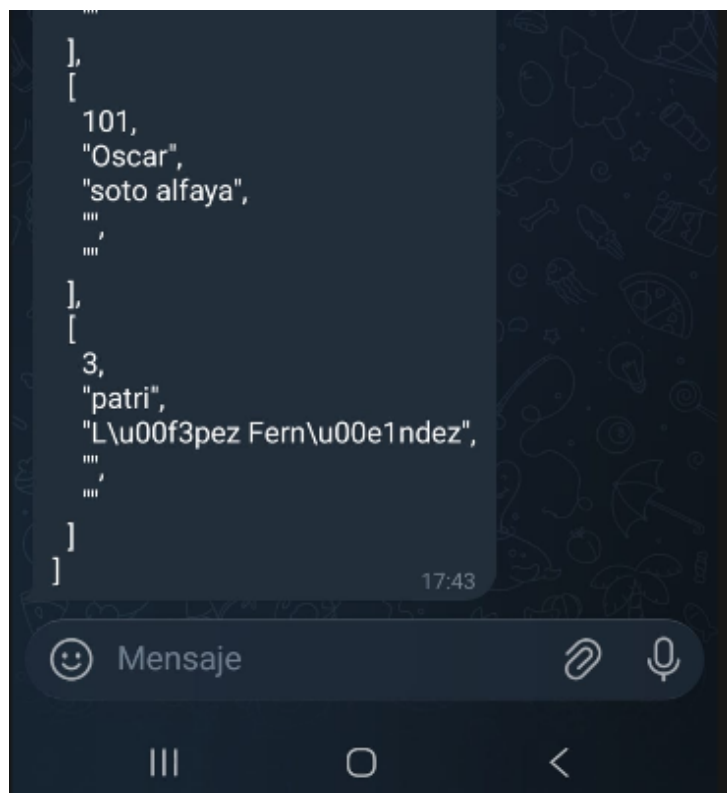


Figura 52: Prueba creaci3n nuevo socio

4. Modificar socio:

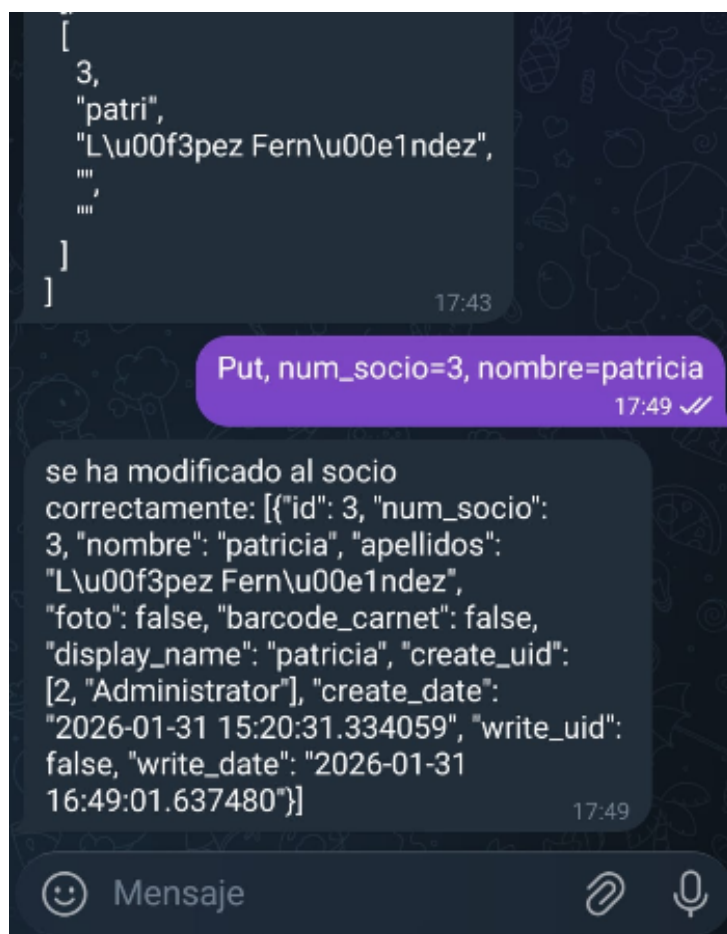


Figura 53: Modificación socio numero 3

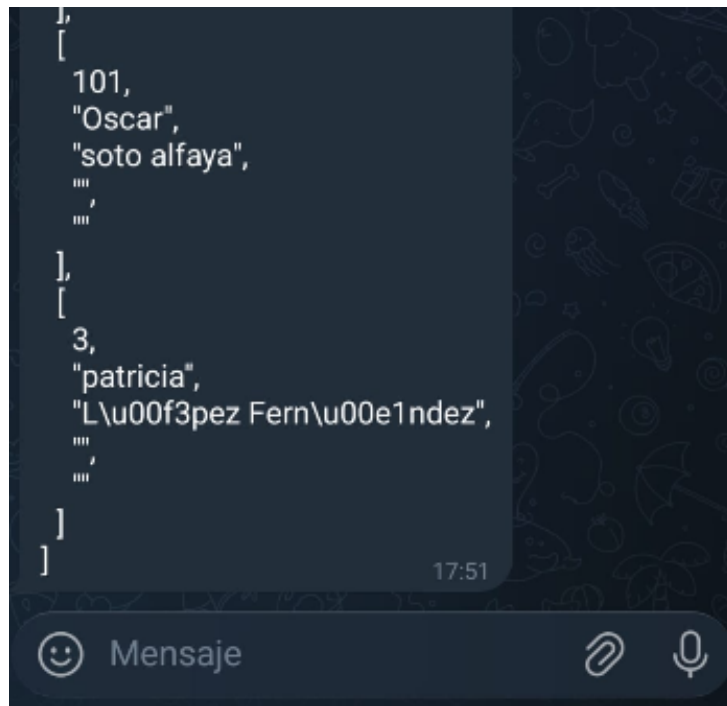


Figura 54: Prueba modificación socio numero 3

5. Borrar socio:

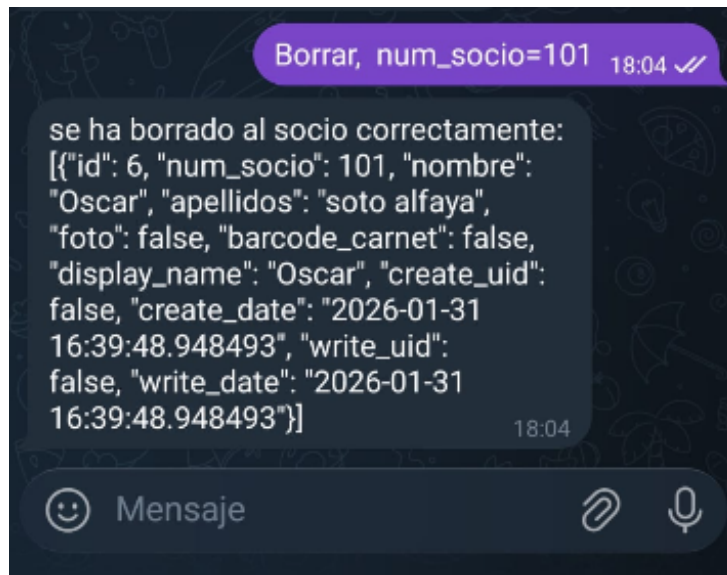


Figura 55: eliminación socio numero 101

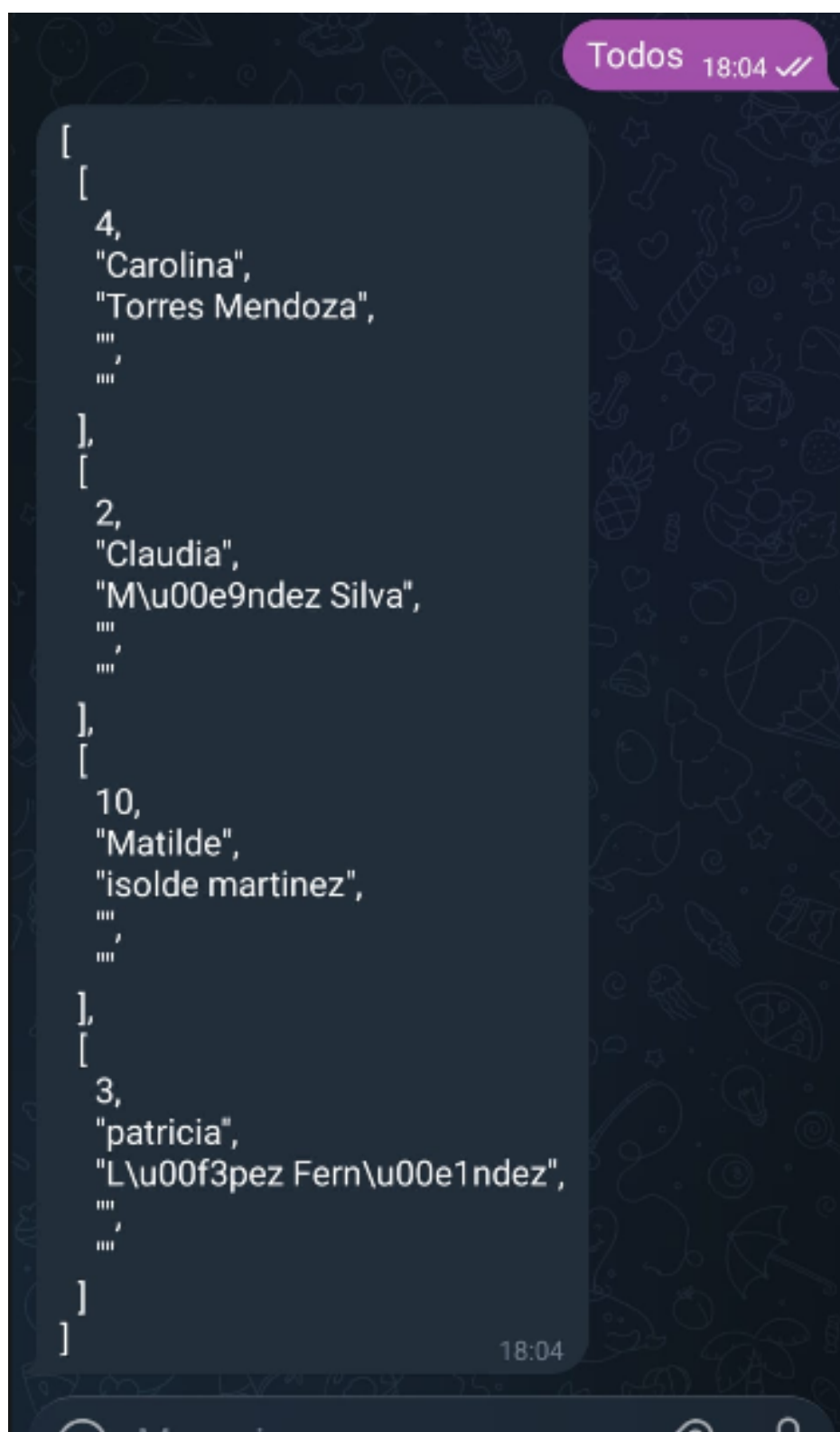


Figura 56: Prueba eliminación socio numero 101

6. LOGS de las operacion en el proyecto

```

PS C:\Users\cuent\Documents\Odoo\Odoo_docker> & C:/Users/cuent/AppData/Local/Programs/Python/Python
Bot SISISI
Se muestran todos los usuarios OOKOKO: <Response [200]>
{'num_socio': '101', 'nombre': 'Oscar', 'apellidos': 'soto alfaya'}
adasdjffffufuf("num_socio": "101", "nombre": "Oscar", "apellidos": "soto alfaya")
Se ha creado al usuario: {'num_socio': '101', 'nombre': 'Oscar', 'apellidos': 'soto alfaya'}
Se muestran todos los usuarios OOKOKO: <Response [200]>
Se ha creado al usuario: {'num_socio': '3', 'nombre': 'patricia'}
Se muestran todos los usuarios OOKOKO: <Response [200]>
Se ha creado al usuario: {'num_socio': '101'}
Se muestran todos los usuarios OOKOKO: <Response [200]>

```

Figura 57: logs en el proyecto

5. Generador de imágenes aleatorias

Para esta parte usaremos el módulo EJ09-GenerarBarcode del repositorio disponible en [este enlace](#). Primero lo probaremos y, posteriormente, necesitaremos instalar la librería Pillow para realizar el ejercicio.

5.1. 4.1 Instalar las librerías

Primero entras en el contenedor de odoo

```
PS C:\Users\ialfper\Desktop\Odoo_docker> docker exec -u root -it f68dfa449315 bash
root@f68dfa449315:/#
```

Figura 58: Entrar al contenedor

Entras al contenedor como root

Código

```
docker exec -u root -it f68dfa449315 bash
```

Meter el comando para instalar barcode o pillow (ojo, pillow por defecto suele venir ya instalado)

```
root@f68dfa449315:/# python3 -m pip install --break-system-packages python-barcode
```

Figura 59: Instalación de librerías

Código

```
python3 -m pip install --break-system-packages python-barcode
```

Para ver las librerías que tienes instaladas en el contenedor de docker de Odoo

```
root@f68dfa449315:/# pip list
WARNING: Skipping /usr/lib/python3.12/dist-packages/charset_normalizer-3.3.2.dist-info due to invalid metadata entry 'name'
Package            Version
-----
asn1crypto         1.5.1
attrs              23.2.0
Babel              2.10.3
beautifulsoup4     4.12.3
cached-property    1.5.2
cbor2              5.6.2
certifi            2023.11.17
chardet            5.2.0
charset-normalizer 3.3.2
cryptography       41.0.7
decorator          5.1.1
defusedxml         0.7.1
docopt             0.6.2
docutils           0.20.1
et-xmlfile         1.0.1
freetype-py        2.4.0
freezegun          1.2.1
geoip2             2.9.0
gevent             24.2.1
```

Figura 60: Ver librerías instaladas

```
ofxparse           0.21
openpyxl           3.1.2
passlib            1.7.4
pdfminer.six       20221105
phonenumbers       8.12.57
pillow             10.2.0
pip               24.0
platformdirs       4.2.0
polib              1.1.1
psutil            5.9.8
pyscoped2          2.9.9
```

Figura 61: Librería pillow

En el caso de que no tengas instalado Pillow puedes instalar las dos a la vez, la de Pillow y barcode.

Código

```
python3 -m pip install --break-system-packages python-barcode pillow
```


5.2. 4.2 Estructura carpeta

Para este ejercicio use la capeta del EJ09-GenerarBarcode del repositorio de ejemplo

En la carpeta de controllers cree otro archivo llamado generarImagenAle.py

```

liga_partido.py  liga_equipo_clasificacion.xml  liga_equipo.xml  liga_partido.xml  main.py  generarbarcode.py 4
addons > EJ09-GenerarBarcode > controllers > generarImagenAle.py > ImagenAleatoria
1  # -*- coding: utf-8 -*-
2  # Importamos clases necesarias de Odoo para definir controladores HTTP
3  from odoo import http
4  import random
5  from PIL import Image
6  import io
7  import base64
8  # Clase controladora para las rutas HTTP que expone nuestro módulo
9  class ImagenAleatoria(http.Controller):
10
11
12      #Se puede probar accediendo a http://localhost:9001/generador/imagenale
13
14      # Ejemplo url http://localhost:9001/generador/imagenaleatoria?ancho=400&alto=400
15      @http.route('/generador/imagenaleatoria', auth='public', cors='*', type='http')
16      def generarImagenAleatoria(self, ancho, alto):
17          #pasar los parametros que mandas a numero
18          ancho=int(ancho)
19          alto=int(alto)
20
21          # creas como la plantilla de la imagen o lienzo, el recuadro de la imagen del tamaño que le pasas
22
23          imagen = Image.new('RGB',(ancho,alto))
24
25          #coger los pixeles de la imagen para luego añadirles color
26          pixeles = imagen.load()
27          #recorrer la altura y lo alto de la imagen y pintar los pixeles de forma aleatoria
28          for eje_x in range(ancho):
29              for eje_y in range(alto):
30                  rojo=random.randint(0,255)
31                  verde=random.randint(0,255)
32                  azul=random.randint(0,255)
33                  pixeles[eje_x, eje_y] = (rojo, verde,azul)
34
35          #guarda la imagen en memoria
36          buffer = io.BytesIO()
37
38          #cuado le des a guardar la imagen te saldra en formato png
39          imagen.save(buffer, format='PNG')
40          # la parte base64.encode convierte bytes binarios en texto seguro que puede ir en URLs/HTML.
41          imagen_convertida = base64.b64encode(buffer.getvalue()).decode('utf-8')
42
43          #devolver la imagen
44          return f''
45
46

```

Figura 62: Código generar imagen aleatoria

Resultado:

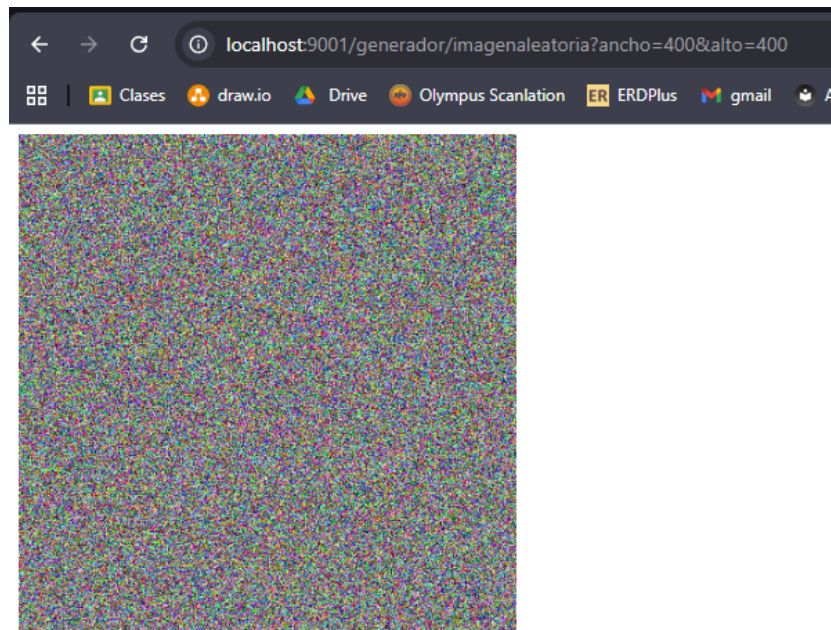


Figura 63: Ejemplo de imagen generada

En el caso de que no pongas un parámetro te dará este error.

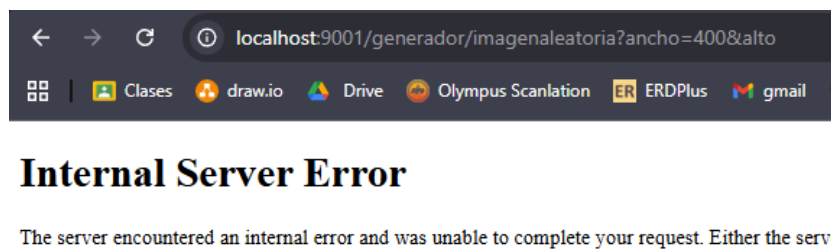


Figura 64: Error sin parámetro